

09Machine-Learning

October 5, 2021

1 Data Science: Bridging Principles and Practice

1.1 Part 9: Machine Learning with Scikit-Learn

In this notebook, we will walk through solving a classification problem using machine learning. To do so, we will introduce the Scikit-Learn machine learning library for Python.

1.1.1 Table of Contents

Case Study: Employee Attrition at IBM

```
<li><a href="#section9">Machine Learning</a>
  <ol type=a>
    <br>
    <li><a href="#section9a">The K-Nearest Neighbors Algorithm</a></li>
    <br>
    <li><a href="#section9b">Using Scikit-Learn: An Example</a></li>
    <br>
    <li><a href="#section9c">Using Scikit-Learn: KNN</a></li>
  </ol>
</li>
</ol>
```

```
In [1]: # run this cell to import some necessary software
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
plt.style.use('fivethirtyeight')
%matplotlib inline
import sklearn
import seaborn as sns
from gofer.ok import check
import warnings
warnings.filterwarnings('ignore')

from sklearn.model_selection import train_test_split
from mpl_toolkits.mplot3d import Axes3D
```

```
# set the random seed for reproducibility
np.random.seed(28)
```

1.2 Case Study: Employee Attrition at IBM

Employee attrition refers to a decrease in employees at a company where the company does not intend to fill the vacancy afterward. Attrition, caused by resignations, retirements, or the elimination of job positions, is important for businesses to anticipate so they can keep costs low and properly distribute workloads.

Today, we'll try to predict whether or not an employee will leave their job using a [data set created by IBM](#). Because employee data is confidential, IBM's data set is fictional- it does not represent real employees. However, the data and the problem are structured very similarly to how actual companies such as IBM are approaching this very real problem.

NOTE: we've made some changes to the data set to make this assignment simpler- things 1.

Run the following cell to load the data.

```
In [2]: # load the data
attrition = pd.read_csv("data/attrition-renamed.csv", index_col="EmployeeNumber")
# show the first 5 rows
attrition.head()
```

```
Out[2]:
```

	Age	Attrition	BusinessTravel	Department
EmployeeNumber				
1	41	1	1	Sales
2	49	0	2	Research & Development
4	37	1	1	Research & Development
5	33	0	2	Research & Development
7	27	0	1	Research & Development

	DistanceFromHome	Education	Gender	JobRole
EmployeeNumber				
1	1	2	1	Sales Executive
2	8	1	0	Research Scientist
4	2	2	0	Laboratory Technician
5	3	4	1	Research Scientist
7	2	1	0	Laboratory Technician

	JobSatisfaction	MaritalStatus	MonthlyIncome
EmployeeNumber			
1	4	Single	5993
2	2	Married	5130
4	3	Single	2090
5	3	Married	2909
7	2	Married	3468

	NumCompaniesWorked	OverTime	PercentSalaryHike
--	--------------------	----------	-------------------

EmployeeNumber

1	8	1	11
2	1	0	23
4	6	1	15
5	1	1	11
7	9	0	12

WorkLifeBalance YearsAtCompany YearsInCurrentRole

EmployeeNumber

1	1	6	4
2	3	10	7
4	3	0	0
5	3	8	7
7	3	2	2

This data set contains a wealth of information that could be used to predict attrition. Here's the **data dictionary**: the information about each feature (column) contained in the data set.

- Age: employee's age
- Attrition: the **response variable**. 1 if employee left IBM due to normal life circumstances, 0 if the employee stayed with IBM
- BusinessTravel: the amount of travel the employee does for business. 0 (no travel), 1 (rare travel), or 2 (frequent travel)
- Department: employee's home department. 'Sales', 'Research & Development', or 'Human Resources'
- DistanceFromHome: number of miles the employee has to travel to work
- Education: employee's highest level of education. 1 (below college), 2 (some college), 3 (bachelor's degree), 4 (master's degree), 5 (doctorate)
- Gender: 1 (female) or 0 (male)
- JobRole: 'Sales Executive', 'Research Scientist', 'Laboratory Technician', 'Manufacturing Director', 'Healthcare Representative', 'Manager', 'Sales Representative', 'Research Director', or 'Human Resources'
- JobSatisfaction: the employee's ranking of their satisfaction with their job. Minimum is 1, maximum is 4.
- MaritalStatus: 'Single', 'Married', or 'Divorced'
- MonthlyIncome: in dollars
- NumCompaniesWorked: the total number of companies the employee has worked at in their lifetime
- OverTime: whether or not the employee is eligible for overtime. 1 (yes) or 0 (no)
- PercentSalaryHike: for the employee's most recent salary increase, the percentage by which their salary increased
- WorkLifeBalance: employee's rating of their work-life balance. 1 (bad), 2 (good), 3 (better), 4 (best)
- YearsAtCompany: number of years employee has worked for IBM
- YearsInCurrentRole: number of years employee has worked for IBM in their current position

1.3 9. Machine Learning

Machine learning is, essentially, pattern-matching. The process involves giving data to a computer, instructing it to find patterns within that data ("learn"), and then to perform a task based on what it has learned. We've already done machine learning in this course, when we trained a linear regression model in notebooks 07 and 08.

The bike-sharing problem from those notebooks and the attrition problem are both examples of **supervised learning**. In both cases, the data used to train the model have *labels*: each row in the data set includes the observed value of the response variable. For example, when we tried to predict bike ridership, we had the actual number of riders. Doing supervised learning is like studying with an answer key- you can use the key to check your answers, which can help you correct your mistakes and learn more quickly and accurately.

There are two main kinds of problems solved in supervised learning: * **Regression problems** predict continuous amounts, as in the bike-sharing problem (predict number of riders based on features like temperature or day of the week). Another example would be predicting the optimal used car price based on its year, make, model, and mileage. * **Classification problems** predict discrete categories or labels, as in our problem today (predicting whether or not an employee will leave a company). Another example would be labeling an image as "cat" or "dog" based on the image features.

<p>

NOTE: the other main category of machine learning is unsupervised learning, where

</p>

<p>Common unsupervised learning algorithms include <i>clustering</i> and <i>neural networks</i>.

</p>

</div>

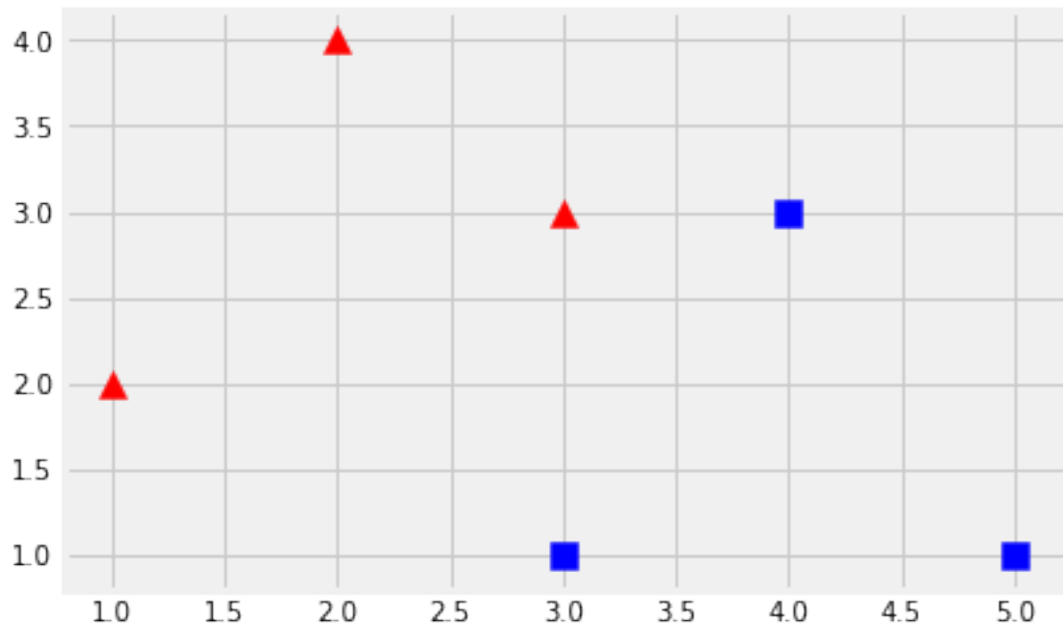
1.3.1 9a. The K-Nearest Neighbors Algorithm

Imagine you are given a simple classification problem. In this problem, there are two possible classes: red triangles and blue squares.

The next cell will create a plot of the data you're given to start out.

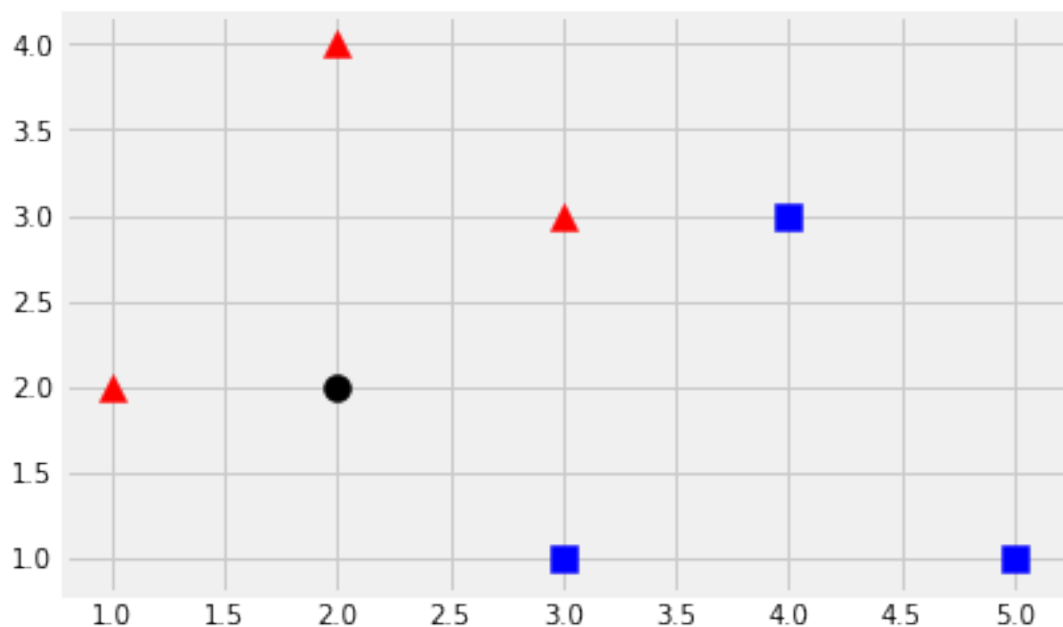
In [3]: # run this cell to show the toy data

```
plt.scatter(x=[1, 2, 3], y=[2, 4, 3], c='r', marker='^', s=100)
plt.scatter(x=[4, 5, 3], y=[3, 1, 1], c='b', marker='s', s=100);
```



Now imagine you are given a new data point to classify. You don't know the correct category, but this new point must be either a red triangle or a blue square. We will represent this unknown new data point with a black circle.

```
In [4]: # add the new data point
plt.scatter(x=[1, 2, 3], y=[2, 4, 3], c='r', marker='^', s=100)
plt.scatter(x=[4, 5, 3], y=[3, 1, 1], c='b', marker='s', s=100)
plt.scatter(x=2, y=2, c='k', s=100);
```



Do you think it's more likely that the new black point is a red triangle or a blue square?

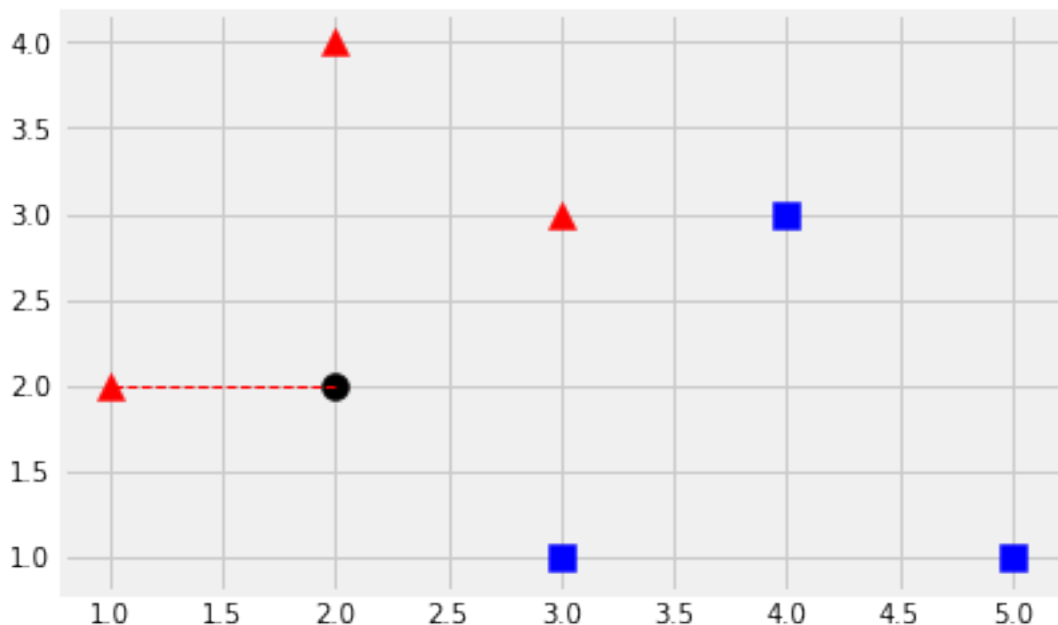
If you're like most people, in the absence of any other information you may have used the **K-Nearest Neighbors** algorithm.

K-Nearest Neighbors (or KNN) picks a class for the new point based on the classes of the closest points- its "neighbors". In this algorithm, "closest" is referring to *Euclidean distance*: the shortest straight-line distance between two points.

The "k" in KNN refers to the number of neighbors that "vote" on the new point's classification. For example, if k=1, the algorithm would find the point closest to the new point...

In [5]: *# show the 1 nearest neighbor*

```
plt.scatter(x=[1, 2, 3], y=[2, 4, 3], c='r', marker='^', s=100)
plt.scatter(x=[4, 5, 3], y=[3, 1, 1], c='b', marker='s', s=100)
plt.plot([2, 1], [2, 2], 'r--', linewidth=1)
plt.scatter(x=2, y=2, c='k', s=100);
```

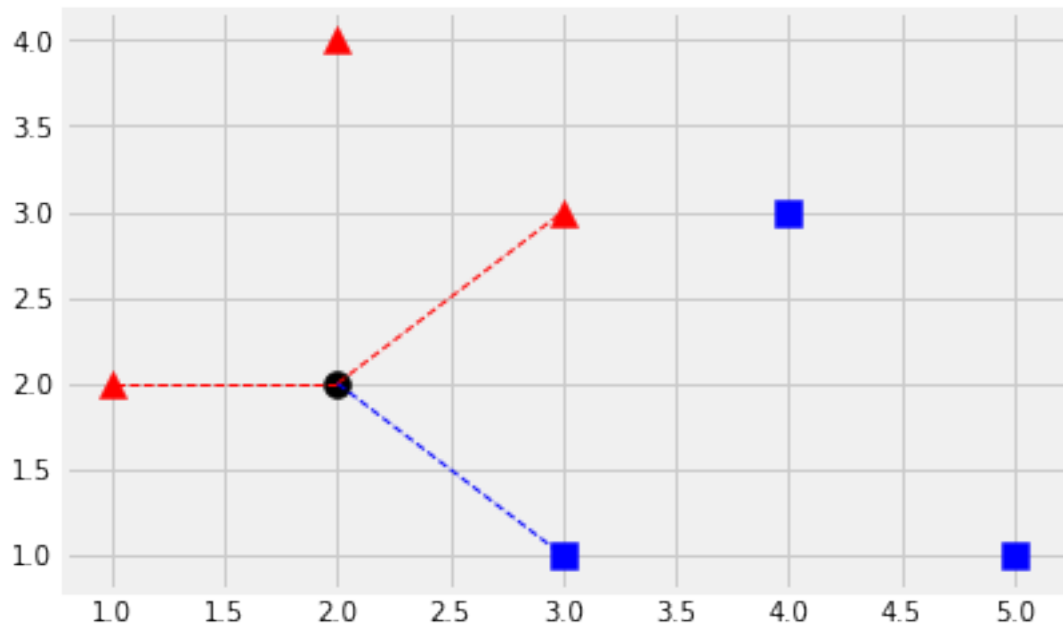


...and conclude that the new point should be a red triangle, because its neighbor is a red triangle.

If instead k=3, the algorithm would look for the three points closest to the new point...

In [6]: *# show the 3 nearest neighbors*

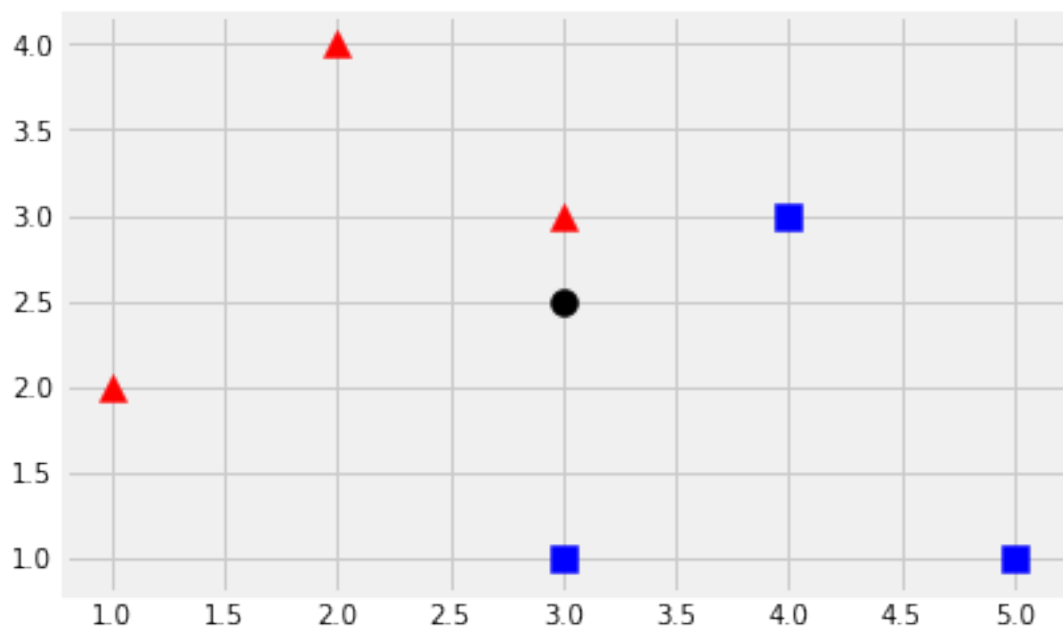
```
plt.scatter(x=[1, 2, 3], y=[2, 4, 3], c='r', marker='^', s=100)
plt.scatter(x=[4, 5, 3], y=[3, 1, 1], c='b', marker='s', s=100)
plt.plot([2, 3], [2, 3], 'r--', linewidth=1)
plt.plot([2, 1], [2, 2], 'r--', linewidth=1)
plt.plot([2, 3], [2, 1], 'b--', linewidth=1)
plt.scatter(x=2, y=2, c='k', s=100);
```



...and if all three points are weighted equally, it would decide the new point is a red triangle since the majority of the three neighbors are red.

The value of 'k' can make a big difference in the accuracy of the algorithm. Imagine you were asked to classify this new black point:

```
In [7]: # add the new data point
plt.scatter(x=[1, 2, 3], y=[2, 4, 3], c='r', marker='^', s=100)
plt.scatter(x=[4, 5, 3], y=[3, 1, 1], c='b', marker='s', s=100)
plt.scatter(x=3, y=2.5, c='k', s=100);
```



QUESTION: Using the KNN algorithm, how would this point be classified for $k=1$, $k=2$, $k=3$?

$k = 1$ - "red" $k = 2$ - "tie" $k = 3$ - "blue"

1.3.2 9b. Using Scikit-Learn: An Example

Scikit-Learn is free, open-source software for machine learning in Python. The software includes tools to create, train, and evaluate machine learning models.

One of the nice things about Scikit-Learn is that the models all have similar frameworks. If you learn how to train a linear regression model, for example, you can train a classification model by using almost identical code. The general steps are:

1. Choose your explanatory variables (what you're going to use to make predictions) and response variable (what you want to predict)
2. Split the data into training and testing sets (and, if you're doing a lot of parameter tuning, a validation set)
3. Create a model in Python
4. Fit the data to the model
5. Make predictions using your fitted model
6. Score the accuracy of your model

In Notebooks 07 and 08, we implemented a linear regression model by writing out most of the code by hand. That's a great way to do things while you're learning, but in the future it would be nice to speed things up. Thankfully, Scikit-Learn can help. In this section, we'll show how to create the linear regression model with Scikit-Learn. In the next section, you'll follow the same steps to create a KNN model.

First, we **choose our explanatory and response variables**. The next cells load the bike data, select "temp", "weather", and "month" as the explanatory variables and "total riders" as the response variable, and create a new X DataFrame and y array that only contain our chosen variables.

```
In [8]: # run this cell to load the data
        bikes = pd.read_csv("data/day_renamed_dso.csv")

        # choose explanatory and response variables
        expl_vars_bikes = ["temp", "weather", "month"]

        resp_var_bikes = "total riders"

In [9]: # create the X DataFrame
        X_bikes = bikes.loc[:, expl_vars_bikes]

        # show the first 5 rows
        X_bikes.head()
```

```
Out [9]:
```

	temp	weather	month
0	0.344167	2	1
1	0.363478	2	1
2	0.196364	1	1
3	0.200000	1	1
4	0.226957	1	1

```
In [10]: # create the y array
        y_bikes = bikes[resp_var_bikes]

        # show the first 5 items
        y_bikes.head()
```

```
Out [10]:
```

0	985
1	801
2	1349
3	1562
4	1600

Name: total riders, dtype: int64

Next, we **split the data into train and test sets**. This code looks a lot like the code from Notebook 07. Running the cell will put 80% of the data into a training set and the remaining 20% into a test set.

NOTE: we're not using a validation set here for simplicity, and because we're not adjusting the model.

```
In [11]: # split the data into training and test sets
        X_train_bike, X_test_bike, y_train_bike, y_test_bike = train_test_split(X_bikes, y_bikes,
                                                                              train_size=0.8)
```

Step three is to **create the model**. The next cell imports the LinearRegression model code from Scikit-Learn, then creates a new model.

A Scikit-Learn model is a Python *object*. To work with a Python object, it can help to conceive of it like a real-life object. Once a real-life object (like a cell phone) is created, it has a set of attributes associated with it (like the manufacturer, model, and storage capacity) and a set of functions it can perform (play music, send email, look up directions...).

Once a Scikit-Learn model is created (on the cloud server), it has a set of attributes associated with it (like the kind of algorithm it uses and whether or not it normalizes data) and a set of functions it can perform (its methods: it can be fitted to a data set, make predictions, and evaluate the accuracy of its predictions). Another example is the DataFrame object that we've been working with for several notebooks. DataFrames have attributes like shape and column names, and they have many functions they can perform, like selecting columns, filtering rows, and manipulating values.

After the model code is imported, we can create a new model. The arguments inside the model creation function call set some of the *hyperparameters*- features of the model that are chosen regardless of the data. In this case, we're using the defaults, which fit an intercept term and don't normalize the training data.

NOTE: we have included the `fit_intercept` and `normalize` arguments here to make it clear that hyperparameters are being set when you create the model. Because we are setting the hyperparameters to their default values, the code in the next cell will create exactly the same base model as if we had included no arguments at all (i.e. if the expression was `LinearRegression()`)

```
In [12]: from sklearn.linear_model import LinearRegression
```

```
lr_model = LinearRegression(fit_intercept=True, normalize=False)
```

Now, the "raw" model can be fit to the training data using the `fit` method. This method takes two arguments: the features in the training X matrix and the labels as listed in the training y array.

```
In [13]: lr_model.fit(X_train_bike, y_train_bike)
```

```
Out[13]: LinearRegression(copy_X=True, fit_intercept=True, n_jobs=None, normalize=False)
```

What just happened?

Behind the scenes, the `fit` method carried out all the steps we went through by hand in Notebook 07: adding an intercept term and using ordinary least squares to calculate β .

Once the model is fit, it has "learned" how to predict total riders based on the features we gave it (month, temperature, and weather). If it is given a matrix of those three features in its `predict` method, it will give back an array of total rider prediction numbers for each row in the matrix.

Using `predict` is the equivalent of taking the dot product by hand, as we did in Notebook 07 using `@`.

```
In [14]: # save the predictions to a variable
lr_predictions = lr_model.predict(X_test_bike)
# show the predictions
lr_predictions
```

```
Out[14]: array([3946.46676074, 5236.27520234, 4269.04963015, 6183.43472695,
6239.02357927, 5307.84743402, 2889.44542339, 4293.64038246,
5075.24404673, 2154.04003282, 2503.39907388, 5802.39957549,
3932.35141846, 4452.47757685, 4361.51730798, 4621.79872315,
5562.55090005, 5266.06866976, 5443.60982694, 4554.75119619,
6764.56775673, 4037.12425325, 5342.54690421, 5814.54232453,
5307.69604584, 4033.04764905, 6253.50561369, 3004.35856306,
2625.30755873, 6445.5288053 , 3500.63341745, 5759.6314826 ,
4077.02037453, 3035.72335521, 4366.56863491, 4240.23694948,
6571.86655474, 6208.02547926, 4545.3204887 , 4360.83929759,
6161.86733444, 4028.36907828, 6491.00893973, 6369.72858124,
6060.1203373 , 2810.97328667, 1902.89685854, 4217.52884016,
6633.18474438, 2213.38994814, 4923.1169764 , 3998.04898865,
4688.47955795, 2139.5579984 , 3052.83548197, 4016.38019528,
5163.33840801, 3640.76912692, 3010.91729895, 3025.39933337,
3166.06166504, 2504.02720024, 2698.28569521, 6128.52388502,
3485.93001505, 2281.04972864, 5539.62748676, 6485.27960242,
2375.96373719, 5130.98428723, 4654.30423211, 6475.17088454,
3151.7310188 , 3245.6922951 , 4517.8638288 , 5254.97668727,
```

```

4907.95693159, 2464.47767533, 3965.84873392, 6895.27882272,
4630.55142025, 6647.6667788 , 5734.36271991, 4037.79372175,
4344.84985421, 2688.58558578, 6531.43774724, 3721.93199615,
5073.05614952, 5650.79912738, 3184.25234693, 5145.31493347,
4229.29883304, 2902.13910435, 1748.47129038, 3765.59092553,
3626.28709249, 2748.5818195 , 3659.47061184, 3358.92031923,
5598.60032699, 4421.852233 , 4209.61160563, 5387.19764007,
5995.1068415 , 5234.75925151, 4901.91627603, 4371.31470761,
6653.39611612, 2350.9692272 , 3642.12514769, 3916.51336326,
4134.48939196, 4651.44080238, 3719.4327807 , 6880.11877791,
4587.4105712 , 3333.89295764, 1481.11127158, 6166.91866137,
3186.2791008 , 5073.05614952, 2944.54778238, 6849.79868829,
3098.60183852, 3931.67340807, 3697.49263203, 4381.4234255 ,
1121.64957666, 2322.98594509, 3112.66429609, 4832.15670753,
4194.45156082, 4502.70378399, 4477.74633954, 6340.086502 ,
4881.18075995, 5726.59935144, 3338.70894749, 4378.44567314,
6203.64609869, 3824.50839186, 5351.8201595 , 2676.56691452,
6278.09030198, 5401.51974444, 3676.82461788])

```

Additionally, our model has a `score` method, which gives information about how well it is doing its prediction task. The `score` method takes as arguments a matrix of feature data (with columns temperature, weather, and month, since those are what it's been trained on) and an array of labels so it can check its answers. The number it returns is a bit different for different types of models. For linear regression, the number is the *coefficient of determination*, aka r^2 . You can double-check what `score` returns by checking out the [Scikit-Learn documentation](#)- scroll down to see the list of methods and read more about `score`.

To sum up, the best possible value of r^2 is 1 and the worst is 0.

```

In [15]: # save the score to a variable
         lr_score = lr_model.score(X_test_bike, y_test_bike)
         # show the score
         lr_score

```

```
Out[15]: 0.47081925298042027
```

1.3.3 9c. Using Scikit-Learn: KNN

Now it's time to return to our IBM employee attrition data set, to see if we can use a k-nearest neighbors model to predict whether or not an employee will leave the company. This section will follow the steps in section 9b very closely- if you're stuck, check the LinearRegression code for some hints.

As a reminder, here's what our attrition data looks like:

```

In [16]: # the first five rows of the attrition data
         attrition.head()

```

```

Out[16]:
   EmployeeNumber  Age  Attrition  BusinessTravel  Department \
1              41    1          1              Sales

```

2	49	0	2	Research & Development
4	37	1	1	Research & Development
5	33	0	2	Research & Development
7	27	0	1	Research & Development

EmployeeNumber	DistanceFromHome	Education	Gender	JobRole \
1	1	2	1	Sales Executive
2	8	1	0	Research Scientist
4	2	2	0	Laboratory Technician
5	3	4	1	Research Scientist
7	2	1	0	Laboratory Technician

EmployeeNumber	JobSatisfaction	MaritalStatus	MonthlyIncome \
1	4	Single	5993
2	2	Married	5130
4	3	Single	2090
5	3	Married	2909
7	2	Married	3468

EmployeeNumber	NumCompaniesWorked	OverTime	PercentSalaryHike \
1	8	1	11
2	1	0	23
4	6	1	15
5	1	1	11
7	9	0	12

EmployeeNumber	WorkLifeBalance	YearsAtCompany	YearsInCurrentRole
1	1	6	4
2	3	10	7
4	3	0	0
5	3	8	7
7	3	2	2

Remember, our steps are:

1. Choose your explanatory variables (what you're going to use to make predictions) and response variable (what you want to predict)
2. Split the data into training and testing sets
3. Create a model in Python
4. Fit the data to the model
5. Make predictions using your fitted model
6. Score the accuracy of your model

Start by choosing explanatory and response variables. For this exercise, we'll try predicting attrition based on **monthly income**, **number of years in current role**, and **overtime eligibility**.

EXERCISE: Choose explanatory and response variables.

explanatory variables must be numerical feature column names that could help predict attrition

the response variable should be the name of the column that says whether or not an employee attrited

</div>

```
In [17]: # choose explanatory and response variables
```

```
expl_vars_attrition = ["MonthlyIncome", "YearsInCurrentRole", "OverTime"]
```

```
resp_var_attrition = "Attrition"
```

```
In [18]: # run this cell to check your answer for some common mistakes
```

```
check("tests/vars.ok")
```

```
Out[18]: <gofer.ok.OKTestsResult at 0x7f52689bd1d0>
```

Now, split the data into training and test sets. If you've chosen your variables correctly, you can just run the next few cells. The training data will be called `X_train_attr` and `y_train_attr`, while the test data will be called `X_test_attr` and `y_test_attr`.

```
In [19]: # once you've chosen your variables, just run this cell
```

```
# create the X DataFrame
```

```
X_attrition = attrition.loc[:, expl_vars_attrition]
```

```
# show the first 5 rows
```

```
X_attrition.head()
```

```
Out[19]:
```

	MonthlyIncome	YearsInCurrentRole	OverTime
--	---------------	--------------------	----------

EmployeeNumber			
----------------	--	--	--

1	5993	4	1
2	5130	7	0
4	2090	0	1
5	2909	7	1
7	3468	2	0

```
In [20]: # once you've chosen your variables, just run this cell
```

```
# create the y array
```

```
y_attrition = attrition[resp_var_attrition]
```

```
# show the first 5 items
```

```
y_attrition.head()
```

```
Out[20]:
```

EmployeeNumber	
----------------	--

1	1
2	0
4	1
5	0
7	0

Name: Attrition, dtype: int64

```
In [21]: # once you've chosen your variables, just run this cell
# split the data into training and test
X_train_att, X_test_att, y_train_att, y_test_att = train_test_split(X_attrition, y_attrition,
                                                                    train_size=0.8, test_size=0.2)
```

The next step is to create the KNN model. The first line of the next cell imports the code for the model, which is called `KNeighborsClassifier`. For now, we'll use the default number of neighbors, which is 5.

EXERCISE: Create the KNN model. Replace the ellipses with a model creation call expression.

```
In [22]: from sklearn.neighbors import KNeighborsClassifier

# create the KNeighborsClassifier
knn = KNeighborsClassifier()
```

```
In [23]: # run this cell to check your answer for some common mistakes
check("tests/model.ok")
```

```
Out[23]: <gofer.ok.OKTestsResult at 0x7f520b6a4160>
```

With the model created, it can now be fit.

EXERCISE: Fit the KNN model to the training data. Use the `fit` method on the model.

```
In [24]: # fit the knn model to the training data
# look at the code for the linear regression model for a hint
knn.fit(X_train_att, y_train_att)
```

```
Out[24]: KNeighborsClassifier(algorithm='auto', leaf_size=30, metric='minkowski',
                             metric_params=None, n_jobs=None, n_neighbors=5, p=2,
                             weights='uniform')
```

A fitted model can make predictions. Use the fitted model's `predict` to predict attrition for the test data. `predict` takes only one argument: an X matrix of feature data with the same features (columns) that it was trained on.

EXERCISE: Use `knn.predict` to make predictions for the test data (`X_test_att`).

```
In [26]: # use the fitted knn model to predict attrition for the test data
# look at the code for the linear regression model for a hint
knn_predictions = knn.predict(X_test_att)
knn_predictions
```

```
Out[26]: array([0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0,
                0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
                0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0,
```

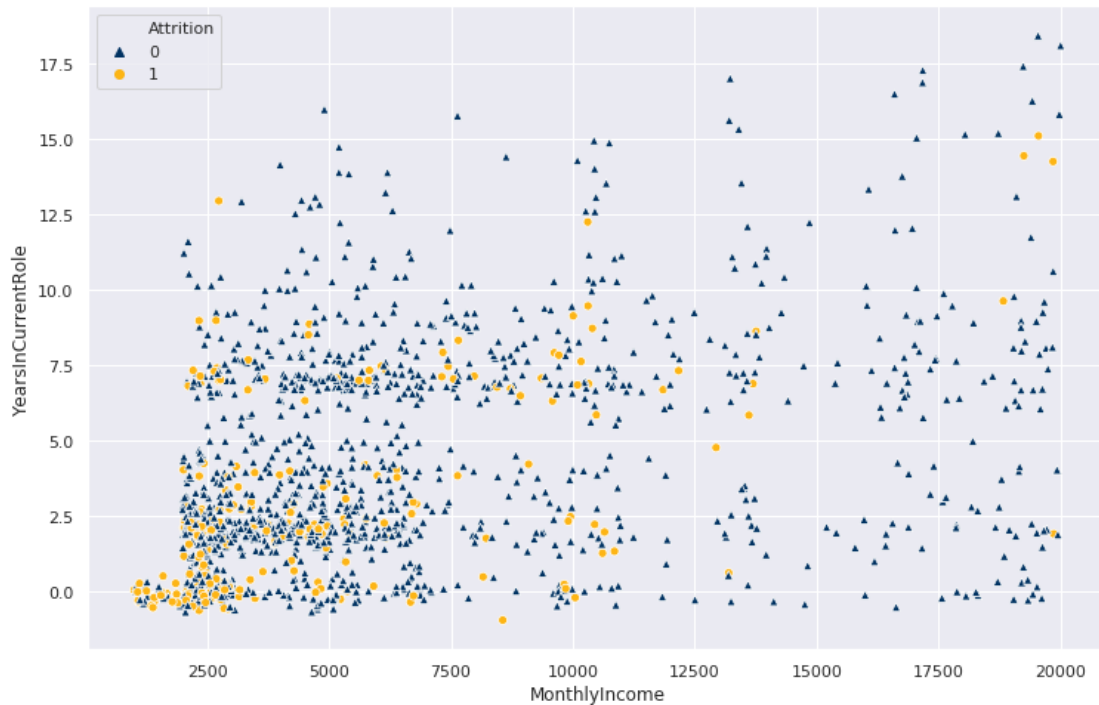


```

In [30]: # make the plots bigger
sns.set(rc={'figure.figsize':(11,8)})

# create the scatter plot
sns.scatterplot(x=attrition["MonthlyIncome"],
                # add a bit of jitter to YearsInCurrentRole so the points overlap less
                y=attrition["YearsInCurrentRole"] + np.random.normal(scale = 0.3, size=
                data=attrition,
                hue="Attrition", style="Attrition", markers={0:"^", 1:"o"}, palette={

```



Sometimes, adding a third dimension can make a difference for the KNN algorithm. The next plot shows all three of our chosen explanatory variables: monthly income, years in current role, and overtime. Note that overtime is a binary value (employees are either eligible for overtime or not), so the overtime axis splits the points into two planes (one for each possible value of "Over-Time").

```

In [31]: # make the 3D scatter plot
fig = plt.figure(figsize=(12, 9))
ax = fig.add_subplot(111, projection='3d')

att_pos = attrition[attrition.Attrition == 1]
att_neg = attrition[attrition.Attrition == 0]

x_label = "YearsInCurrentRole"
y_label = "MonthlyIncome"

```

```

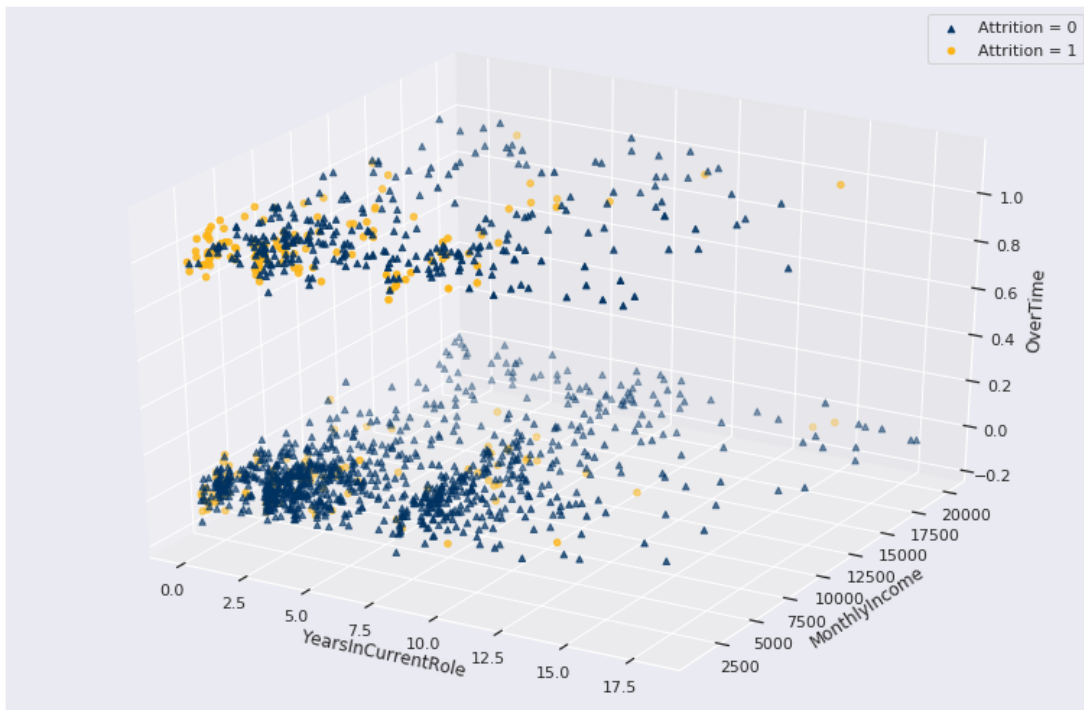
z_label = "OverTime"

for data, m , l, c in [(att_neg, "^", "Attrition = 0", "#003262"), (att_pos, "o", "Attrition = 1", "#FFA500")]:
    ax.scatter(data[x_label], data[y_label], (data[z_label] + np.random.normal(scale=0.1, loc=0)),
               marker=m, label=l, color=c)

ax.set_xlabel(x_label)
ax.set_ylabel(y_label)
ax.set_zlabel(z_label)

plt.legend();

```



QUESTION: based on the model score, how well is our classifier working? Based on the score, what can we conclude about the model's performance?

YOUR ANSWER HERE

References

- IBM HR Analytics Employee Attrition & Performance mock data set is made available under the [Open Database License](#). Any rights in individual contents of the database are licensed under the [Database Contents License](#).